Germain D. McCaulay-Jones

Email | LinkedIn | GitHub | Portfolio

Summary

Software engineer focused on backend systems and data-intensive applications - open to fintech, full-stack, and systems/HPC roles.

Skills

Languages: Python, C++, Java, TypeScript, SQL, Bash
Frameworks: Pandas, NumPy, TensorFlow, FastAPI, UnitTest, MPI
Tools: Git, GitHub Actions, Docker, Linux/Unix, AWS, GCP, PostgreSQL, Crow
Competencies: Design Patterns, Version Control, System Design, Agile, Test-Driven Development, Data Engineering Lifecycle, Relational Databases, CI/CD

Education

University of Bath 2021 – 2025

Master of Computing in Computer Science and Mathematics with Honours (2:1)

- **Relevant Modules:** Stats for Data Science, Machine Learning, Data Structures & Algorithms, Functional Programming, Parallel Programming, Graphs & Networks, Linear & Abstract Algebra, Real & Complex Analysis, Differential & Geometric Analysis

The Warriner School 2014 – 2021

A-Levels and GCSEs

- Further Mathematics ($A*$), Mathematics ($A*$), Physics ($A*$), Computer Science (A)
- GCSEs in 10 subjects (including English and Maths)
- Additional Mathematics (A) [**FSMQ**]

Projects

Traveler Backend API

C++, PostgreSQL, Crow, libpqxx, Docker, GoogleTest, golang-migrate, JWT, Firebase, AWS

- Built a stateless backend API in **C++ (Crow)** for journey management, with clear separation between domain models, application services, and web-layer request/response contracts.
- Implemented PostgreSQL persistence using **libpqxx** with parameterised queries and a repository pattern to keep the core logic database-agnostic.
- Managed schema evolution with **golang-migrate** and reproducible local development via a containerised **Postgres** setup.
- Added **JWT** authentication middleware to secure routes while maintaining a stateless architecture.
- Wrote automated tests with **GoogleTest** to validate core behaviours and API functionality.

Age Prediction Website

Python, TypeScript, Bash, TensorFlow, FastAPI, GCP, Docker, GitHub Actions

- **Shipped an end-to-end age prediction web app** with a typed front end, containerised inference API, and automated CI/CD to **Cloud Run** for reproducible deployment.
- Trained and evaluated two models on **UTKFace (5,000 images)**: a custom CNN (∼0.5M params) and an Xception transfer-learning model (7.3M trainable params), achieving **7.56** vs. **6.88** years validation MAE.
- Implemented a **React + TypeScript** UI with a typed API contract, request/response validation, and CI via **GitHub Actions** to enforce repeatable builds and tests on every change.
- Containerised **TensorFlow** and **FastAPI** services with automated build-and-push via **Bash**.

Variational Quantum Eigensolver Development (Master's Dissertation)

Python, IBM Qiskit, Pandas, unittest, NumPy, Matplotlib

- **Built a reproducible benchmarking and evaluation pipeline** for hybrid VQA/VQE-style factorisation of RSA semiprimes in **Qiskit**, comparing encodings and backends under controlled experimental settings.
- Extended a hybrid VQA/VQE factoriser, evaluating **five** cost-function encodings across **two** circuit families, and structured experiments to make comparisons consistent across runs (fixed parameter initialisation, sweep configs, and logged artefacts).
- Quantified NISQ sampling trade-offs via controlled shot sweeps. For $n = \mathbf{2047}$ in noiseless simulation, success increased from **0.00** (1k shots) to **0.50** (5k shots), indicating a clear measurement threshold effect.
- Implemented a **unittest** suite for experiment components (encoding selection, backend config, result parsing) and produced analysis plots and tables using **Pandas** and **Matplotlib** for transparent reporting.

Parallel Matrix Smoothing Coursework

C, MPI, Pthreads

- **Implemented a high-performance parallel matrix relaxation solver** in **C**, delivering both **Pthreads** (shared memory) and **MPI** (distributed memory) versions with validated numerical equivalence to a sequential baseline.
- Designed the distributed approach using domain decomposition and **non-blocking halo exchange**, overlapping communication with computation to reduce synchronisation overhead on matrices up to 40,000 × 40,000.
- Improved throughput by eliminating $\mathcal{O}(n^2)$ copies via pointer swapping and reducing contention using per-thread convergence tracking, informed by profiling and scaling experiments.
- Measured strong-scaling performance, reducing runtime from **56.8s (10 MPI processes)** to **6.84s (170 MPI processes)**, and documented experiment conditions and timing methodology for repeatability.

Soct Programming Language (Bachelor's Dissertation)

Java, ANTLR4, unittest

- **Built an interpreted toy language (Soct)** inspired by the Solos calculus, implementing a full pipeline from parsing to execution to model concurrent process interactions.
- Used **ANTLR4** to generate a lexer and parser, constructing an AST and a Visitor-driven IR builder with scoped channel resolution, consistent error reporting, and repeatable CLI execution.
- Implemented a runtime that resolves communications and performs name substitution using equivalence-class style unification, with a focus on correctness and deterministic reductions.
- Added **unit tests** covering parsing, scoping, and representative reduction programs, including negative tests for malformed syntax and invalid channel usage.

Hobbies & Interests

- Avid reader across software, mathematics, political theory, and poetry.
- Personal tooling on Arch Linux, including ricing and workflow automation (dotfiles, scripts).
- Powerlifting and calisthenics.
- Competitive gaming, previously competing with Vulpine Esports.